SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 10%

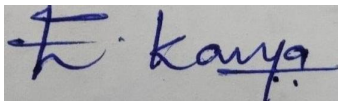
Duration :60 Mins
On Class
QUESTIONS: 50 **Total Marks:50**

Annexure A

SCENARIO BASED TASK

Code of Conduct:

I **E.Kavya(192424365)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

SCENARIO BASED TASK

Q1. A compiler development team is implementing a simple desk-calculator module that evaluates arithmetic expressions entered by users. The calculator supports addition, subtraction, multiplication, division, parentheses, and single-digit operands. The compiler uses a Syntax-Directed Definition (SDD) to compute the value of each expression during parsing.

The grammar used by the calculator is:

S → E n

E → E + T
  | E - T
  | T

T → T * F
  | T / F
  | F

F → (E)
  | digit

For the input expression:

(6 + 4) * (4 + 3)

Analyze the grammar and design an appropriate SDD for evaluating arithmetic expressions. Construct the annotated parse tree for the given input and show the attribute values computed at each relevant node. Finally, determine the final value produced by the calculator.

Q2. A compiler development team is testing the syntax-directed translation module of an arithmetic expression processor. During testing, the expression

a * b - c + 8

is given as input. The compiler must correctly represent the expression according to operator precedence and associativity and use a Syntax-Directed Definition (SDD) to evaluate the expression.

Analyze the given expression, construct its syntax tree considering the precedence and associativity of the operators, and formulate an appropriate SDD for evaluating the expression. Show the attribute evaluation sequence for the given expression.

Q3.A compiler development team is designing a semantic-analysis module that uses inherited and synthesized attributes to pass information between grammar symbols during translation. The team needs to verify whether the semantic rules used in the grammar satisfy the conditions of an L-attributed definition, so that the attributes can be evaluated from left to right during parsing.

Consider the production rules:

$A \rightarrow P\ Q$
$A \rightarrow X\ Y$

Each of the non-terminals A, P, Q, X, and Y has two attributes:

- s – synthesized attribute
- i – inherited attribute

The compiler team proposes the following semantic rules:

Rule 1:

$P.i = A.i + 2$
$Q.i = P.i + A.i$
$A.s = P.s + Q.s$

Rule 2:

$X.i = A.i + Y.s$
$Y.i = X.s + A.i$

Analyze the two sets of semantic rules with respect to the conditions of an L-attributed definition. Determine which rule satisfies the requirements of an L-attributed definition and which rule violates them. Justify your answer by examining the dependency of inherited and synthesized attributes and their left-to-right evaluation order.

Q4. A compiler development team is implementing an expression translator for a calculator application. The translator must evaluate arithmetic expressions and generate an appropriate intermediate code representation using an S-attributed Syntax-Directed Definition (SDD). The syntax analyzer uses LR parsing to process the input from left to right and construct the required parse structure.

The compiler uses the following grammar:

$S \rightarrow E\ N$
$E \rightarrow E + T$
$E \rightarrow E - T$
$E \rightarrow T$
$T \rightarrow T * F$
$T \rightarrow T / F$
$T \rightarrow F$
$F \rightarrow (E)$

F → digit
N → ;

The input expression provided to the translator is:

2 + 5 * 6 ;

Analyze the given grammar and design an S-attributed Syntax-Directed Definition for generating the corresponding code fragment (translator). Then trace the LR parsing process for the input string 2 + 5 * 6 ; using a parser stack, showing the sequence of shift and reduce operations. Finally, show the generated translation for the given input.

Q5.A compiler development team is implementing a top-down expression translator for an arithmetic calculator. The translator must process arithmetic expressions containing addition, multiplication, parentheses, and single-digit operands. During translation, the compiler should use a Syntax-Directed Definition (SDD) to compute the value of the expression and annotate the corresponding parse tree.

The grammar used by the translator is:

S → E n

E → E + T
  | T

T → T * F
  | F

F → (E)
  | digit

The compiler is required to process the following input:

3 * 4 + 8 n

Analyze the given grammar and formulate a suitable SDD for top-down translation. Construct the annotated parse tree for the given input expression, showing the attribute values at the appropriate nodes. Finally, trace the top-down evaluation and determine the final value generated by the translator.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

CO3 - AT3 [Scenario Based]

Name : E. Kavya

RegNo : 192424365

Code : CSA1408

Course : Compiler design

1) SDD for Evaluating Arithmetic Expressions

Aim: To design a Syntax-Directed Definition (SDD) for evaluating arithmetic expressions and construct the annotated parse tree for (6+4) * (4+3).

Grammer

S → En

E → E+T / E-T / T

T → T * F / T / F / F

F → (E) / digit

Syntax-Directed Definition

Use the synthesized attribute .val

S → En { S.val = E.val }

E → E1 + T { E.val = E1.val + T.val }

E → E1 - T { E.val = E1.val - T.val }

E → T { E.val = T.val }

T → T1 * F { T.val = T1.val * F.val }

T → T1 / F { T.val = T1.val / F.val }

T → F { T.val = F.val }

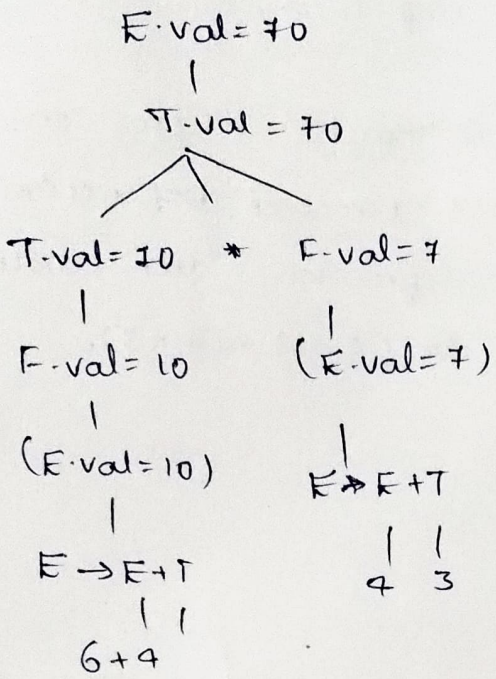
F → (E) { F.val = E.val }

F → digit { F.val = digit.lexical }

-Annotated parse tree

-for

(6+4) * (4+3)



Important attribute calculations

6+4 = 10, 4+3 = 7, 10*7 = 70

Attribute Evaluation

F·val = 6
F·val = 4
E·val = 6+4 = 10

F·val = 4
F·val = 3
E·val = 4+3 = 7

T·val = 10*7 = 70
E·val = T·val = 70

final result

(6+4) * (4+3) = 10 * 7 = 70

final value = 70

2) Syntax tree and SDD for a*b-c+8

Aim: To construct the syntax tree according to operator precedence and associativity and formulate an SDD for evaluating the expression.

Expression

a*b - c +8

Operator rules

↠ * has higher precedence than - and +.

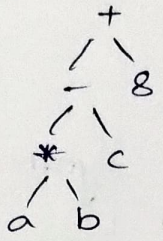
→ - and + have the same precedence

→ They are evaluated from left to right.

Therefore

((a*b) - c)+8

Syntax tree



So the expression is

((a*b) - c)+8

SDD

$E \rightarrow E_1 + T$ {E.val = E1.val + T.val}

$E \rightarrow E_1 - T$ {E.val = E1.val - T.val}

$E \rightarrow T$ {E.val = T.val}

$T \rightarrow T_1 * F$ {T.val = T1.val * F.val}

$T \rightarrow F$ {T.val = F.val}

$F \rightarrow id$ {F.val = id.val}

Attribute Evaluation

a*b → T.val = a*b

(a*b) - c → E.val = (a*b) - c

((a*b) - c)+8 → E.val = (a*b - c) + 8

Result

The correct expression structure is

((a*b) - c)+8

3) L-Attributed Definition

Aim: To determine whether the given semantic rules satisfy the conditions of an L-attributed definition.

Production

$A \rightarrow P\ Q$

$A \rightarrow X\ Y$

Each symbol has synthesized attribute S and inherited attribute I.

Rule 1:

$P.I = A.I + 2$

$Q.I = P.I + A.I$

$A.S = P.S + Q.S$

Analysis

→ P.I depends on A.I → allowed

→ Q.I depends on P.I, which belongs to the symbol on its left → allowed

→ A.S depends on synthesized attributes of P & Q

∴ Rule 1 is L-attributed.

Rule 2

$X.I = A.I + Y.S$, $Y.I = X.S + A.I$

for $A \rightarrow XY$

X.I depends on Y.S

But Y is to the right of X. Therefore X.I cannot depend on an attribute of a symbol to its right in an L-attributed definition.

∴ Rule 2 is not L-attributed.

4) S- Attributed SDD and LR Parsing

Aim: To design an s-attributed SDD and trace the LR Parsing of 2+5*6;

Grammer

S → EN

E → E+T | E-T | T

T → T*F | T/F | F

F → (E) | digit

N → ;

SDD : Use synthesized attribute .place

E → E1+T {E.place = newtemp(); emit(E.place = E1.place + T.Pal}

T → T1*F { T.Pal = newtemp(); emit(T.pal = T1.place * F.Pal); }

E → T { E.pal = T.pal }

T → F { T.pal = F.pal }

F → digit { F.Pal = digit.lexval }

LR shift / Reduce Summary

2 → Shift → Reduce F → Reduce T → Reduce E + → shift

5 → shift → Reduce F → Reduce T

* → shift

6 → shift → Reduce F

Reduce T → T*F

Reduce E → E+T ; → shift → Reduce N → Reduce S

Accept

Generated code

Because * has heghes precedence

$t_1 = 5*6 = 30$

$t_2 = 2+t_1 = 32$

Result

Generated entermediate code is

$t_1 = 5*6$, $t_2 = 2+t_1$ final value = 32

5) Top-Down Expression Translation

Aim:

To formulate an SDD and evaluate the expression

3*4+8n.

Grammes

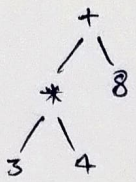
S → En

E → E+T | T

T → T*F | F

F → (E) | digit

Annotated tree



SDD

S → En {S.val = E.val}

E → E1+T {E.val = E1.val + T.val}

E → T {E.val = T.val}

T → T1*F {T.val = T1.val * F.val}

T → F {T.val = F.val}

F → (E) {F.val = E.val}

F → digit {F.val = digit.lexval}

Attribute Evaluation

3 → F.val = 3 3*4 = 12

4 → F.val = 4

8 → T.val = 8

12+8 = 20

Therefore → S.val = 20

Result

The expression is evaluated as:

(3*4) + 8 = 20

final value = 20